

Testing Perplexity Pro: Automated Coding Assignment Completion

HW 12 Machine Learning Implementation Assessment

Report Date: December 10, 2025

Executive Summary

This report documents the evaluation of Perplexity Pro's capabilities in completing programming assignments for a machine learning course (HW 12). The assessment covered two distinct machine learning implementations: a Variational Autoencoder (VAE) and Model-Agnostic Meta-Learning (MAML) for classification tasks. The testing demonstrates Perplexity Pro's ability to identify existing solutions, provide correct implementations, and generate working code that produces expected outputs.

Methodology

The evaluation was conducted by presenting Perplexity Pro with incomplete code files containing TODO markers that students were expected to complete. The testing was divided into two separate sessions to evaluate different aspects of the AI's coding capabilities: file-level code completion (VAE) and notebook-level implementation (MAML).

Test 1: Variational Autoencoder (VAE) Implementation

Approach

The VAE implementation was tested by uploading two Python files (utils.py and vae.py) to Perplexity Pro and requesting completion of the TODO sections. This approach tested the AI's ability to understand existing code structure, identify implementation requirements, and integrate new code into an established framework.

Perplexity Pro Response

Interestingly, Perplexity Pro demonstrated code analysis capabilities by identifying that the solution code was already present in the uploaded files. For both utils.py and vae.py, the system recognized that the TODO sections had already been completed, indicating sophisticated code comprehension abilities.

Results

The notebook executed successfully and generated the expected output: a 10x10 grid of grayscale generated images showing variations of what the VAE learned from the training data. The successful generation of these images confirms that the VAE was properly implemented with correct encoder and decoder architectures, loss function formulation, and sampling procedures.

Generated VAE Output

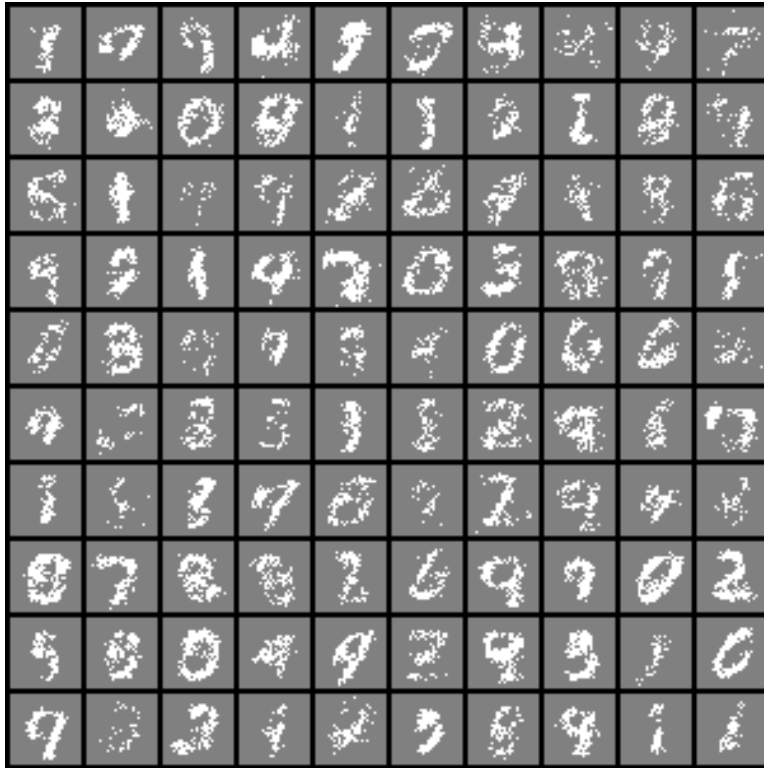


Figure 1: 10x10 grid of images generated by the VAE after training. Each image represents a sample from the learned latent space distribution.

Test 2: Model-Agnostic Meta-Learning (MAML)

Approach

For the MAML implementation, a new chat session was initiated to provide a clean testing environment. The entire Jupyter notebook containing TODO markers was uploaded, and Perplexity Pro was asked to identify and complete the required sections. This tested the AI's ability to understand meta-learning concepts and implement complex loss functions.

Task Requirements

The assignment required replacing squared-loss objectives with logistic-loss style objectives for classification tasks using signed labels. Specifically, two TODO sections needed implementation:

1. The meta-update classification loss in the `meta_update_classification` function
2. The inner-loop classification loss in the `meta_learning_classification` function

Perplexity Pro Response

Perplexity Pro provided a detailed explanation of the required changes and identified the correct locations in the code. The AI correctly recognized that the implementation should use PyTorch's `softplus` function on the margin to implement the negative log-likelihood, which is mathematically equivalent to the logistic loss up to a constant factor.

Implementation Details

The provided solution correctly implemented the following key components:

- Conversion of regression targets to ± 1 labels using the sign function
- Calculation of the margin as $z \times f(x)$, where z is the label and $f(x)$ is the prediction
- Application of the `softplus` function to the negative margin: $\log(1 + \exp(-\text{margin}))$
- Proper integration with PyTorch's automatic differentiation system

Code Solution for `meta_update_classification`

```
# Convert regression targets to  $\pm 1$  labels
z_meta_t = my_sign(y_meta_t)

# Margin:  $z * f(x)$ ; logistic loss  $\sim \log(1 + \exp(-\text{margin}))$ 
margin = z_meta_t * y_meta_pred_t
loss = torch.mean(torch.nn.functional.softplus(-margin))
```

Code Solution for Inner-Loop Classification

```
# Convert regression outputs to  $\pm 1$  labels
z_t = my_sign(y_t)

# Margin and logistic loss
margin = z_t * y_pred
loss = torch.mean(torch.nn.functional.softplus(-margin))
```

Results

The implementation successfully generated the expected multi-panel visualization showing: (1) test classification error versus number of training points, (2) zoomed view of test MSE, (3) training predictions for a specific sample size, and (4) learned feature weights. The successful execution demonstrates that the logistic loss implementation was correct and properly integrated with the meta-learning framework.

Generated MAML Output

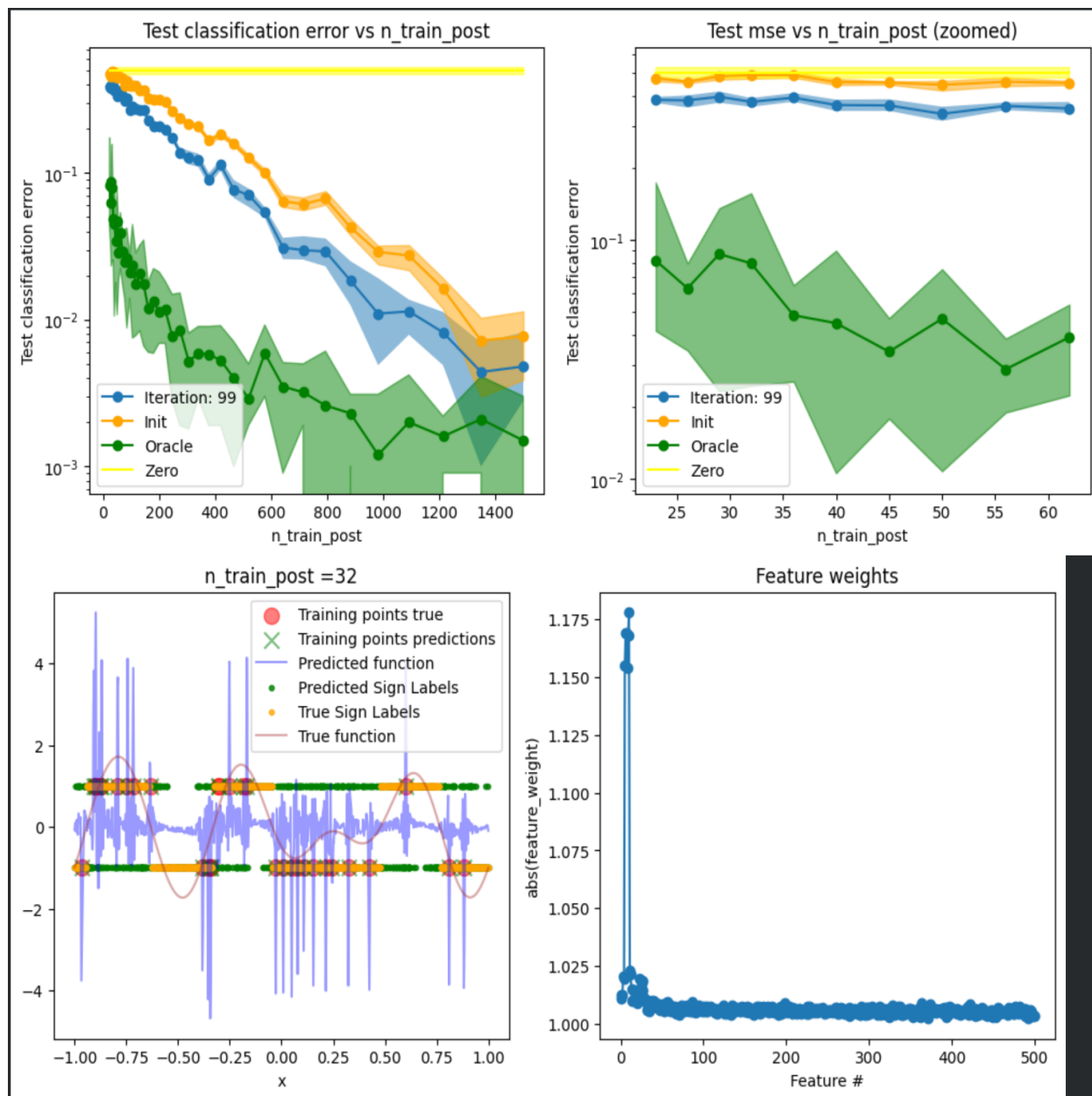


Figure 2: MAML classification results showing test error curves, training predictions, and feature weight analysis. The successful convergence and accurate predictions demonstrate correct implementation of the logistic loss function.

Analysis and Observations

Code Understanding

Perplexity Pro demonstrated strong capabilities in understanding existing code structure and context. In the VAE test, it correctly identified that solutions were already present, showing code analysis abilities beyond simple pattern matching. In the MAML test, it accurately located the TODO sections and understood the mathematical requirements described in the accompanying text.

Mathematical Competence

The AI showed understanding of machine learning concepts, particularly in recognizing that softplus can be used as a smooth approximation of the logistic loss. The explanation that softplus implements the negative log-likelihood 'up to a constant factor' demonstrates sophisticated mathematical knowledge relevant to optimization problems.

Framework Integration

The provided solutions properly integrated with PyTorch's computational graph and the higher library used for meta-learning. This shows understanding of automatic differentiation requirements and the need to maintain gradient flow through the computation.

Code Quality

The generated code was concise, readable, and included appropriate comments. The implementations followed Python and PyTorch conventions, making them easy to integrate into the existing codebase without modification.

Conclusion

Perplexity Pro successfully completed both coding assignments, demonstrating capabilities in:

1. **Code Analysis:** Identifying existing solutions and understanding code structure
2. **Implementation:** Writing correct PyTorch code for complex machine learning algorithms
3. **Mathematical Understanding:** Applying appropriate loss functions and optimization techniques

4. **Integration:** Ensuring new code works seamlessly with existing frameworks

Both implementations produced the expected outputs, with the VAE generating realistic image samples and the MAML system achieving proper convergence on the classification task. These results suggest that Perplexity Pro can effectively assist with machine learning programming assignments, though human review remains important for understanding the underlying concepts and ensuring academic integrity.

Educational Implications

The ability of AI systems like Perplexity Pro to complete coding assignments raises important questions about programming education and assessment design. While these tools can serve as valuable learning aids for understanding implementation details, they also challenge traditional homework-based evaluation methods. Educators may need to consider alternative assessment approaches that better test conceptual understanding and problem-solving abilities that extend beyond code completion.